

SUPERCUTE: Measuring Exact Character Perception and Long-Workflow State Tracking in Language Models

Stanislav Kirdey
Clark Labs Inc.
stan@clarkslabs.com

June 2026

Abstract. SUPERCUTE measures two things about language models. First, how well they perceive exact character-level details such as Unicode normalization, homoglyphs, zero-width characters, and grapheme boundaries. Second, how reliably they track and update precise state in long real-world workflows, like replaying dozens of CRM record edits or applying sequential legal redlines to a contract. Every task is generated and graded by deterministic Python code; no model ever judges another model’s answer. We evaluate six reasoning-enabled models on three scored tiers: character perception probes, realistic workflows, and a controlled long-horizon stress test. We also release PublicLift, an unscored set of templates that turn public-dataset records into fresh exact-state tasks. The character probes turn out not to be the hard part: GPT-5.5 scores 149/152 on homoglyphs, zero-width characters, Unicode normalization, code points, UTF-8 byte length, and grapheme operations. Doing real work with those characters is harder: GPT-5.5 scores 50/64 on the workflow tier, and the weakest model scores 14/64. The controlled `iterated_lut` tier shows the sharpest split: GPT-5.5 falls from 7/8 at 330 dependent updates to 0/8 at 1320 updates, while Claude Opus 4.8 scores 33/40 overall and goes 8/8 at the same 1320-update setting. So there is no universal frontier wall here. The honest claim is narrower: seeing tricky characters is largely solved for GPT-5.5, but carrying exact state through hundreds of small dependent changes fails in high-variance, model-specific ways that deserve larger sweeps and token-budget-controlled comparisons.

1 Introduction

One wrong character can ruin a day of work. A single invisible character can change a customer ID. A Cyrillic letter that looks like a Latin one can split an invoice group in two. A line number in a patch shifts after one edit, and every later edit lands in the wrong place. A contract amendment can point at a clause whose identifier looks right but is not. Earlier work such as CUTE [1] made the perception half of this problem concrete by asking whether language models can reason about the characters inside their tokens, and CharBench [2] shows that character counting and position tasks remain hard for many models.

This paper asks a plain question: what still goes wrong when a frontier reasoning model is allowed to think at length? Our answer is deliberately modest. In our run, GPT-5.5 has essentially learned to see: isolated character probes are nearly saturated. What it has not reliably learned is to keep its hands steady through a long job — replaying many small edits where each one depends on the ones before, with no shortcut and an exact final answer. And even that failure is not universal: on our controlled lookup-table task, GPT-5.5 collapses at longer horizons while Opus 4.8 barely slows down. The interesting signal is model-specific reliability, not a single leaderboard number.

SUPERCUTE keeps the whole evaluation auditable. Every task has ground truth computed by deterministic

code. The grader accepts exact answers and extracts final answers from verbose responses. There is no manual judging in the core score, and no task is hard only because it secretly requires a tool, such as computing a cryptographic hash.

We make four contributions:

- A negative result: character perception is near-saturated for GPT-5.5 on our battery.
- RealTok: eight workflows modeled on real jobs — CRM edit replay, invoice cleanup, legal redline merges, warehouse reconciliation — where the score is the exact final state.
- A controlled pilot break curve for shortcut-free full-state execution, plus a simple compounding-error fit.
- PublicLift: templates that turn finance tables, contracts, receipts, logs, code traces, software patches, and spreadsheets into fresh exact-state tasks. These templates are not scored in this run.

2 Related Work

Tokenizer and character understanding. CUTE tests whether models can manipulate characters despite operating over tokens [1]. CharBench scales this idea and separately studies occurrence tracking and positional understanding [2]. Byte-level or patch-level architectures such as BLT attempt to reduce dependence on fixed token boundaries [3]. SUPERCUTE asks the next question:

when a reasoning model can already pass isolated character probes, where does it still fail in realistic work?

Long-horizon execution. Several recent studies argue that long tasks fail because small errors pile up. The Illusion of Diminishing Returns shows how single-step quality turns into task-length limits [4]. Work on finite-state-machine execution isolates state maintenance over long horizons [5]. Long-context benchmarks such as RULER and NoLiMa show that finding a fact in a long context is not the same as working with it [6, 7]. SUPERCUTE focuses on exact mutable state rather than broad QA.

Real-world substrates. Public datasets offer realistic records and automatic or semi-automatic oracles. FinQA and TAT-QA use financial reports and tables [9, 10]; TabFact and WikiTableQuestions use semi-structured tables [11, 12]; SpreadsheetBench is built from real spreadsheet problems [13]; CRUXEval tests code execution [14]; SWE-bench evaluates patches for real GitHub issues [15]; CUAD and ContractNLI cover legal contracts [16, 17]; CORD and DocVQA cover receipts and documents [18, 19]; LogHub collects real system logs [20]. PublicLift uses these datasets as raw material, not as static questions.

3 Benchmark Design

SUPERCUTE uses generators rather than fixed puzzles. A generator receives a random seed and returns a prompt, answer, kind, and metadata. This gives fresh variants, exact ground truth, and cheap self-checking.

3.1 Core requirements

A task is included only if it meets four requirements.

- **Exact oracle.** The answer is computed by deterministic code.
- **No hidden tool dependency.** A model should not need a calculator, code interpreter, or web browser.
- **No easy shortcut in the hard tier.** If a model can skip most of the history — because only the last write matters, or a formula gives the answer — the task is a diagnostic, not a breaker.
- **Realistic contact point.** In RealTok and PublicLift, the hard detail should look like work a person might hand off: merging redlines, replaying patches, cleaning receipts, reconciling ledgers, auditing logs.

3.2 Scored tiers

Tok. The character perception tier includes 19 tasks: can the model spot a Cyrillic look-alike letter, find a zero-width character hiding in an ID, tell two Unicode normalization forms apart, report code points and UTF-8 byte

lengths, catch confusable digits, and segment graphemes correctly?

RealTok. RealTok includes eight workflows modeled on real jobs: reconstructing a CRM field from a support agent’s cursor edits, applying a ticket-board JSON patch stream, cleaning a Unicode-noisy invoice CSV, auditing synthetic redaction leaks, merging contract redlines, reconciling warehouse barcode scans, reconstructing a clinical sequence, and merging localization string tables. The score is always the exact final state.

LUT. The `iterated_lut` tier is a controlled stress test. It samples a random 10×10 transition table T and a digit string s of width $L = 22$. Each round applies

$$s'_i = T[s_i][s_{i+1}],$$

with wraparound. We vary the number of rounds $K \in \{15, 30, 45, 60, 75\}$, giving 330 to 1650 dependent cell updates. Because T is random and the whole state changes every round, there is no known closed form and no way to skip ahead. Since L is fixed in this run, this is a depth sweep, not a two-dimensional width-by-depth study.

PublicLift release tier. PublicLift is released infrastructure, not scored evidence in this paper. Each template takes a public record — a financial table, a contract, a receipt, a log — and generates a fresh seeded changelog against it. The public record supplies realism; the fresh changelog defeats memorization; a Python oracle supplies the exact label. The templates do not redistribute third-party rows.

4 Experimental Setup

We evaluated six reasoning-enabled models through OpenRouter in June 2026: GPT-5.5, Claude Opus 4.8, DeepSeek V4 Pro, MiniMax M3, Qwen 3.7 Plus, and Qwen 3.5 Flash. Temperature was set to 0. The Tok and RealTok tiers use $n = 8$ variants per task. The LUT break curve uses $n = 8$ variants for each model and each value of K . We report exact-match accuracy and Wilson 95% confidence intervals.

The raw run contains 1536 model calls. Thirteen provider or transport errors are kept in the raw JSONL but dropped from scored denominators, leaving 1523 scored rows. This explains denominators such as 33/62 and 4/37. Tables report scored denominators, and the release preserves the dropped-error counts.

The grader first looks for explicit answer markers and otherwise extracts the final answer-like span from the end of the response. This matters because reasoning models often explain at length before answering. We validated the extractor against raw model outputs before scoring. A

Model	Correct	Acc.	95% CI
GPT-5.5	149/152	0.980	[0.944, 0.993]
DeepSeek V4 Pro	136/152	0.895	[0.836, 0.934]
Qwen 3.7 Plus	131/152	0.862	[0.798, 0.908]
MiniMax M3	129/152	0.849	[0.783, 0.897]
Opus 4.8	128/152	0.842	[0.776, 0.892]
Qwen 3.5 Flash	121/152	0.796	[0.725, 0.852]

Table 1: Character perception tier. These probes separate weaker models from stronger ones, but they do not break GPT-5.5 in this run.

Model	Correct	Acc.	95% CI
GPT-5.5	50/64	0.781	[0.666, 0.865]
DeepSeek V4 Pro	33/62	0.532	[0.410, 0.651]
Opus 4.8	33/63	0.524	[0.403, 0.642]
MiniMax M3	25/61	0.410	[0.295, 0.535]
Qwen 3.7 Plus	20/60	0.333	[0.227, 0.459]
Qwen 3.5 Flash	14/64	0.219	[0.135, 0.334]

Table 2: RealTok tier: character-exact details embedded in realistic jobs. Denominators below 64 reflect dropped provider/transport errors.

free self-check verifies that every generator’s own answer is accepted by both the strict and robust grader.

Reasoning budget is a major confound. Models expose and spend reasoning tokens differently, and some providers report hidden reasoning separately while others mainly expose total completion tokens. We therefore report token counts for the LUT tier and trust within-model curves more than cross-model rankings. Even the within-model curve is a pilot, because $n = 8$ per cell is small.

5 Results

5.1 Finding 1: GPT-5.5 can see the characters

Table 1 shows the Tok tier. GPT-5.5 solves 149 of 152 items. Its few misses are not in Unicode perception at all; they are in long-run counting and similar static exactness tasks. Tricky characters on their own — look-alike letters, invisible characters, normalization traps — are not enough to break the strongest model in our run.

5.2 Finding 2: doing real work with those characters is harder

Table 2 shows the RealTok tier. The same character details become much harder when they sit inside a job with state that keeps changing. GPT-5.5 remains best but drops to 50/64. Qwen 3.5 Flash falls to 14/64.

The hardest RealTok tasks are the edit-replay ones. Re-

Model	Correct	Acc.	95% CI
Opus 4.8	33/40	0.825	[0.680, 0.913]
DeepSeek V4 Pro	16/40	0.400	[0.263, 0.554]
GPT-5.5	15/40	0.375	[0.242, 0.530]
Qwen 3.5 Flash	11/40	0.275	[0.161, 0.428]
MiniMax M3	10/40	0.250	[0.142, 0.402]
Qwen 3.7 Plus	4/37	0.108	[0.043, 0.247]

Table 3: Overall LUT accuracy across $K \in \{15, 30, 45, 60, 75\}$. Qwen 3.7 Plus has three dropped provider/transport errors, giving 37 scored calls.

constructing a CRM field from a stream of cursor edits scores 2/8 for GPT-5.5 and 0/8 for every other model. Merging sequential legal redlines scores 3/8 for GPT-5.5, 2/8 for Opus, and 0/8 for the rest. This fits a simple picture: moving cursors, shifting positions, and exact text preservation give a model many small chances to slip, and one slip is enough.

One RealTok task is intentionally instructive: `json_patch_ticket_board` is easy for every model. Most of its fields are last-write-wins, so a model can ignore most of the history and still be right. That is a design warning for benchmark builders: if the history can be compressed, it will be.

5.3 Finding 3: long horizons break models one at a time, not all at once

Figure 1 and Table 3 show the LUT break curve. GPT-5.5 falls sharply as the number of dependent updates grows: 7/8 at $K = 15$, 4/8 at $K = 30$, 3/8 at $K = 45$, 0/8 at $K = 60$, and 1/8 at $K = 75$. These are complete answers with normal stop reasons, not truncated outputs. The model finishes the job; it just loses the state along the way.

The curves are not monotonic, and the cells are small. GPT-5.5 goes from 0/8 at $K = 60$ back to 1/8 at $K = 75$, while Opus dips to 6/8 at $K = 45$ and recovers to 8/8 at $K = 60$. So the right reading is not “longer always means worse” from these $n = 8$ cells. It is that shortcut-free full-state execution produces noisy but visible reliability differences between models.

The result is also entangled with how much thinking each model does. At $K = 60$, GPT-5.5 averages about 16.8k completion tokens and scores 0/8. Opus averages about 34.8k completion tokens and scores 8/8. GPT-5.5’s token use rises from $K = 15$ to $K = 45$ and then nearly plateaus, while Opus keeps writing more as K grows. But spending more is not enough by itself: Qwen 3.5 Flash reaches about 87k completion tokens at $K = 75$ and still scores 0/8. The safer conclusion is that token budget and per-step precision interact.

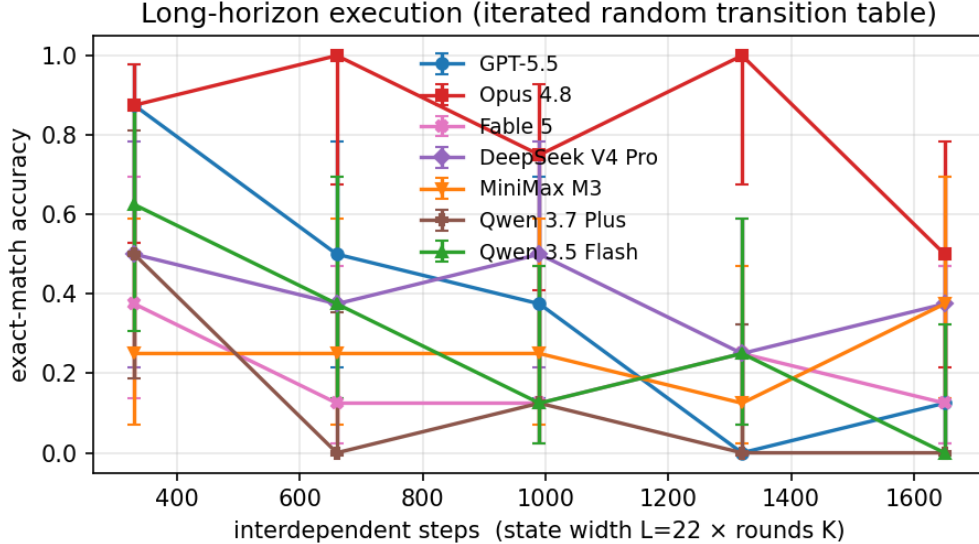


Figure 1: Long-horizon execution pilot for `iterated_lut`. The x-axis is $22 \times K$ dependent updates. Error bars are Wilson 95% intervals. The jagged curves show why larger n is needed before drawing fine-grained ordering claims.

Model	r	q	half upd.
Opus 4.8	0.9956	0.999800	3459
DeepSeek V4 Pro	0.9790	0.999034	717
GPT-5.5	0.9739	0.998798	576
MiniMax M3	0.9690	0.998572	485
Qwen 3.5 Flash	0.9659	0.998422	439
Qwen 3.7 Plus	0.9359	0.996993	230

Table 4: Descriptive fit of $P(\text{correct}|K) = r^K$ on the LUT tier. Here r is fitted per-round survival, q is equivalent per-update survival, and “half upd.” is the fitted half-life in cell updates.

5.4 A simple compounding-error fit

If small execution errors pile up, final exact accuracy should decay with task length. To summarize the jagged curves, we fit the one-parameter model

$$P(\text{correct} \mid K, m) = r_m^K = q_m^{22K},$$

where r_m is fitted per-round survival for model m and q_m is the equivalent per-cell-update survival under fixed $L = 22$. This is a descriptive binomial maximum-likelihood fit, not evidence that errors are independent.

The fit makes the model-specific story concrete. Opus has a fitted half-life of about 3459 updates on this task, six times GPT-5.5’s 576, which matches its 33/40 overall score. GPT-5.5 is not the best model on LUT despite being strongest on Tok and RealTok. That alone rules out any claim that all frontier models share the same long-horizon wall.

6 PublicLift: Using Public Data to Go Harder

Public datasets are useful, but reusing their original questions is not. Many original labels are short, possibly memorized, or answerable by lookup. PublicLift uses a procedural lift instead:

public record + fresh changelog $\rightarrow$ new exact-state task.

The changelog is generated at evaluation time. It contains realistic operations, Unicode-confusable identifiers, and many state updates. The oracle replays the same operations in Python and emits the final answer. In this paper PublicLift is a release contribution and a future evaluation direction, not a scored result.

The lift makes each task look like the job a person would actually delegate. A finance task becomes “apply this quarter’s restatements and roll the segment totals forward,” not a single table question. A contract task becomes “merge these five amendments and show me the final clause,” not clause classification. A receipt task becomes OCR cleanup plus reconciliation, not one-field extraction. A software task becomes “replay these patches and tell me what is on each line now,” not issue summarization. PublicLift gives the benchmark a path from controlled synthetic stress tests to human-relevant work, but it still needs a paid multi-model sweep before becoming an empirical claim.

7 Code Release

The release is a normal Python package: a registry of scenario generators, an OpenAI-compatible API client,

PublicLift task	Public substrate family	Exact target
<code>finqa_restatement_rollforward</code>	FinQA, TAT-QA, MultiHiertt	final segment totals in cents
<code>tabfact_table_changelog</code>	TabFact, WikiTableQuestions, SpreadsheetBench	final requested rows
<code>cuad_amendment_merge</code>	CUAD, ContractNLI, LEDGAR	final clause text
<code>cord_receipt_reconciliation</code>	CORD, SROIE, DocVQA	final category totals
<code>loghub_incident_replay</code>	LogHub and AIOps logs	sorted open incidents
<code>cruxeval_stack_trace</code>	CRUXEval, LiveCodeBench, CodeNet	final stack
<code>swebench_patch_manifest</code>	SWE-bench and software patches	final line snapshots
<code>spreadsheet_cell_reconciliation</code>	SpreadsheetBench	final cell values and checksums

Table 5: PublicLift templates released with SUPERCUTE. These are designed to be fed by public records without redistributing third-party data.

strict and robust graders, sweep and benchmark runners, public-data adapter skeletons, analysis scripts, tests, CI, and documentation. The package needs nothing beyond the Python standard library at runtime. Optional extras support plotting, datasets, and development.

The commands are deliberately short. Run `python -m supercute.selfcheck` for the full local oracle check. Module-level self-checks use `python -m supercute.sweep -module <name> -self`, where `<name>` can be `tok`, `realtok`, `hard`, or `publiclift`. A paid model sweep only requires an OpenAI-compatible endpoint and API key. The full evaluation harness can optionally write raw prompts, oracle answers, and model responses for manual failure analysis.

8 Limitations and Next Experiments

These results come from one provider route and one time window. Reasoning models are not fully deterministic, even at temperature 0. Reasoning budgets differ across models and endpoints, so cross-model rankings should be read alongside token spend. The LUT cells use only $n = 8$, which makes the break curves jagged and the Wilson intervals wide. A convincing follow-up should use at least 30 to 50 variants per cell.

The LUT sweep fixes width $L = 22$ and varies depth K , so it cannot say whether width or depth matters more; a two-dimensional $L \times K$ grid is needed. We also have no human baseline. That matters because RealTok tasks are framed as delegated work, and the benchmark should verify that careful humans can do representative items reliably given enough time. Finally, exact-match grading is intentionally strict, and the scored JSONL from this run does not include raw responses, so this paper cannot offer a full failure taxonomy. The release adds an optional raw-output audit path for future runs.

9 Conclusion

SUPERCUTE started from a suspicion that tokenizers were still the weak point, and the data said otherwise. GPT-5.5 can see homoglyphs, invisible characters, and normalization traps almost perfectly. What it cannot always do is the long, careful part of real work: replaying dozens of CRM edits, merging sequential redlines, or carrying a 22-digit state through 1320 dependent updates without dropping it. And even that weakness is model-specific — Opus 4.8 handles the same long task far better. The practical lesson for benchmark builders: measure exact state updates, remove shortcuts, report denominators and token budgets, and treat long-horizon execution as a per-model reliability question rather than a frontier-wide wall.

Reproducibility Statement

The released artifact contains source code, raw results, summary tables, the figure generator, tests, CI configuration, and this LaTeX source. Core tasks are deterministic and self-checkable without model calls. The raw results include provider/transport errors so denominator choices can be audited.

References

- [1] Edman, L., Schmid, H., and Fraser, A. CUTE: Measuring LLMs’ understanding of their tokens. EMNLP, 2024.
- [2] Uzan, O. and Pinter, Y. CharBench: Evaluating the role of tokenization in character-level tasks. AAAI, 2026.
- [3] Pagnoni, A. et al. Byte Latent Transformer: Patches scale better than tokens. arXiv:2412.09871, 2024.
- [4] Herel, D. et al. The illusion of diminishing returns: Measuring long-horizon execution in LLMs. arXiv:2509.09677, 2025.
- [5] Anonymous. The illusion of procedural reasoning: Evaluating LLMs’ long-horizon execution of finite state machines. arXiv:2511.14777, 2025.

- [6] Hsieh, C.-P. et al. RULER: What’s the real context size of your long-context language models? COLM, 2024.
- [7] Keren, N. et al. NoLiMa: Long-context evaluation beyond literal matching. arXiv:2502.05167, 2025.
- [8] Kulagin, E. et al. TokenBreak: Advancing text classification attack by token manipulation. arXiv:2506.07948, 2025.
- [9] Chen, Z. et al. FinQA: A dataset of numerical reasoning over financial data. EMNLP, 2021.
- [10] Zhu, F. et al. TAT-QA: A question answering benchmark on a hybrid of tabular and textual content in finance. AAAI, 2021.
- [11] Chen, W. et al. TabFact: A large-scale dataset for table-based fact verification. ICLR, 2020.
- [12] Pasupat, P. and Liang, P. Compositional semantic parsing on semi-structured tables. ACL, 2015.
- [13] Ma, Z. et al. SpreadsheetBench: Towards challenging real world spreadsheet manipulation. NeurIPS, 2024.
- [14] Gu, A. et al. CRUXEval: A benchmark for code reasoning, understanding and execution. ICML, 2024.
- [15] Jimenez, C. E. et al. SWE-bench: Can language models resolve real-world GitHub issues? ICLR, 2024.
- [16] Hendrycks, D., Burns, C., Chen, A., and Ball, S. CUAD: An expert-annotated NLP dataset for legal contract review. NeurIPS Datasets and Benchmarks, 2021.
- [17] Koreeda, Y. and Manning, C. D. ContractNLI: A dataset for document-level natural language inference for contracts. Findings of EMNLP, 2021.
- [18] Park, S. et al. CORD: A consolidated receipt dataset for post-OCR parsing. NeurIPS Workshop on Document Intelligence, 2019.
- [19] Mathew, M., Karatzas, D., and Jawahar, C. V. DocVQA: A dataset for VQA on document images. WACV, 2021.
- [20] Zhu, J. et al. Loghub: A large collection of system log datasets for AI-driven log analytics. ISSRE, 2023.